

<pre>#ejemplo de tipos numericos nombre = "juan" saludo = "hola, "+ nombre print (saludo) #salida: hola, juan</pre>	hola, juan
<pre># eljemplos de tipos numericos a = 10 # enterp b = 3.14 # punto flotante c = a + b print (c) # salida: 13.14</pre>	13.14
<pre>print(f"juan recibe despues de decir {saludo} un vuelto de {c}")</pre>	juan recibe despues de decir hola, juan un vuelto de 13.14
<pre>import math print(math.pi)</pre>	3.141592653589793
<pre>import math print(f"{math.pi:.3f}")</pre>	3.142
<pre>print("=== lea con cuidado lo que se pide ===") A = input ("ingrea el numero entero deseado de cifras significativas: ") A=int(A) import math print(f"el numero pi con {A} cifras decimales es igaul a: {math.pi:.{A}f}") B=4 C=A+B print(C)</pre>	=== lea con cuidado lo que se pide === ingrea el numero entero deseado de cifras significativas: 10 el numero pi con 10 cifras decimales es igaul a: 3.1415926536 14
<pre>def saludar(nombre): return f"hola, {nombre} eres una chica formidable" # llamada a la funcion nombre = input("ingrsa tu nombre:") mensaje = saludar(nombre) print(mensaje) #hola, nombre</pre>	ingrsa tu nombre:majo hola, majo eres una chica formidable
<pre>print("=== Operaciones Basicas ===") print(f"Suma: 2 + 3 = {2 + 3}") print(f"Resta: 10 - 4 = {10 - 4}") print(f"Multiplicacion: 6 * 7 = {6 * 7}") print(f"Division: 15 / 3 = {15 / 3}") print(f"Potencia: 2 ** 8 = {2 ** 8}") print(f"Modulo (Residuo): 17 % 5 = {17 % 5}")</pre>	=== Operaciones Basicas === Suma: 2 + 3 = 5 Resta: 10 - 4 = 6 Multiplicacion: 6 * 7 = 42 Division: 15 / 3 = 5.0 Potencia: 2 ** 8 = 256 Modulo (Residuo): 17 % 5 = 2
<pre>def sofia_suma(a,b): return f"esta es la suma del {a} y del {b} cuyo resultado es: {a+b}" sofia_suma(2,1)</pre>	'esta es la suma del 2 y del 1 cuyo resultado es: 3'
<pre>def majo_resta(a,b): return f"esta es la resta del {a} y del {b} cuyo resultado es: {a-b}" majo_resta(2,1)</pre>	'esta es la resta del 2 y del 1 cuyo resultado es: 1'
<pre>def majo_multiplicacion(a,b): return f"esta es la multiplicacion del {a} y del {b} cuyo resultado es: {a*b}" majo_multiplicacion(2,1)</pre>	'esta es la multiplicacion del 2 y del 1 cuyo resultado es: 2'
<pre>def majo_division(a,b): return f"esta es la division del {a} y del {b} cuyo resultado es: {a/b}" majo_division(2,1)</pre>	'esta es la division del 2 y del 1 cuyo resultado es: 2.0'

```
def majo_potencia(a,b):  
    return f"esta es la potencia del {a} y del {b} cuyo resultado es: {a**b}"  
  
majo_potencia(2,1)  
  
'esta es la potencia del 2 y del 1 cuyo resultado es: 2'
```

```
def majo_residuo(a,b):  
    return f"este es el residuo del {a} y del {b} cuyo resultado es: {a%b}"  
  
majo_residuo(15,5)  
  
'este es el residuo del 15 y del 5 cuyo resultado es: 0'
```

```
# funcion para hallar el lado de un cuadrado, conociendo su lado  
def area_cuadrado(lado):  
    return f"el area del cuadrado de lado {lado} es: {lado**2} unidades cuadradas"  
  
area_cuadrado(5)  
  
'el area del cuadrado de lado 5 es: 25 unidades cuadradas'
```

```
# funcion para hallar el area ed un rectangulo  
def area_rectangulo(base,altura):  
    return f"el area del rectangulo de base {base} y altura {altura} es: {base*altura} unidades cuadradas"  
  
area_rectangulo(5,10)  
  
'el area del rectangulo de base 5 y altura 10 es: 50 unidades cuadradas'
```

Regiones Básicas de la Geometría Plana y su Cálculo de Área

Departamento de Matemáticas

6 de agosto de 2026

Resumen

Este artículo presenta una revisión concisa de las regiones planas elementales: triángulo, cuadrado, rectángulo, rombo, paralelogramo, trapecio, círculo y sector circular. Para cada una se define su área mediante fórmulas geométricas, se interpreta su significado y se señalan las relaciones entre sus dimensiones características. El enfoque está orientado a facilitar la comprensión intuitiva y la aplicación práctica en problemas de medición.

1. Introducción

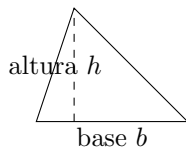
El cálculo de áreas es una de las herramientas más antiguas y fundamentales de la geometría. La noción de área como la medida de la superficie encerrada por una curva cerrada permite cuantificar regiones del plano. En este trabajo se clasifican las figuras planas según su número de lados y simetrías, y se deriva la expresión de su área a partir de propiedades elementales.

2. Triángulo

El triángulo es la región plana delimitada por tres segmentos de recta. Su área es la mitad del producto de una base por su altura correspondiente:

$$A = \frac{1}{2}bh.$$

Esta fórmula se obtiene al completar un paralelogramo de igual base y altura, cuya área es bh , y observar que el triángulo ocupa exactamente la mitad.



3. Cuadrado

El cuadrado es un cuadrilátero regular con cuatro lados iguales y ángulos rectos. Su área es simplemente el cuadrado de su lado:

$$A = l^2.$$

Deriva de la definición de área como producto de la base por la altura, que coinciden en el cuadrado.

4. Rectángulo

El rectángulo es un paralelogramo con ángulos rectos, cuyos lados opuestos son iguales. Su área es el producto de su base por su altura:

$$A = b \cdot h.$$

Es la figura que da origen a la noción de área en coordenadas cartesianas.

5. Rombo

El rombo es un paralelogramo equilátero (cuatro lados iguales) pero no necesariamente equiángulo. Su área se calcula como el producto de sus diagonales dividido por dos:

$$A = \frac{D \cdot d}{2},$$

donde D y d son las diagonales mayor y menor. También puede expresarse como $A = l^2 \sin \theta$, siendo θ uno de sus ángulos internos.

6. Paralelogramo

Cualquier paralelogramo (no necesariamente rectángulo ni rombo) tiene área igual al producto de una base por la altura perpendicular a esa base:

$$A = b \cdot h.$$

Esta fórmula se demuestra por descomposición en un rectángulo equivalente mediante un corte y traslación de un triángulo.

7. Trapecio

El trapecio es un cuadrilátero con al menos un par de lados paralelos (bases). Su área es el promedio de las bases multiplicado por la altura:

$$A = \frac{(B + b)}{2} \cdot h.$$

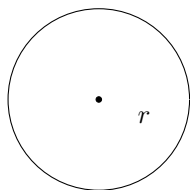
Intuitivamente, es el área de un rectángulo cuya base es la longitud media de las bases paralelas.

8. Círculo

El círculo es el conjunto de puntos del plano que distan menos o igual que un radio r de un punto central. Su área es:

$$A = \pi r^2.$$

Esta expresión se obtiene mediante el límite de polígonos regulares inscritos o mediante integración en coordenadas polares.



9. Sector circular

Un sector circular es la porción de círculo comprendida entre dos radios y el arco que los une. Su área es proporcional al ángulo central θ (en radianes):

$$A = \frac{1}{2}r^2\theta.$$

Cuando $\theta = 2\pi$, se recupera el área del círculo completo.

10. Relaciones y generalizaciones

Todas estas fórmulas son casos particulares del *teorema de Green* o de la integral de línea, pero en el plano elemental se deducen por descomposición en rectángulos, triángulos o por métodos de exhaustión. La elección de la fórmula adecuada depende de los datos disponibles (lados, alturas, diagonales, ángulos o radio).

11. Aplicaciones prácticas

El cálculo de áreas es esencial en topografía, arquitectura, diseño de superficies y en la resolución de problemas de optimización. Conocer las relaciones entre las dimensiones permite, por ejemplo, maximizar el área para un perímetro dado (caso del círculo) o comparar regiones de formas distintas.

Conclusiones

Hemos recorrido las regiones planas más básicas y sus respectivas fórmulas de área. Cada una surge de principios geométricos simples y se extiende a figuras más complejas mediante descomposición. La comprensión de estas regiones es la base para abordar el cálculo de áreas en curvas arbitrarias y en sólidos de revolución.

Referencias

- [1] Euclides, *Elementos*, Libros I–VI.
- [2] R. Pérez, *Geometría Plana y Trigonometría*, Editorial Universitaria, 2010.
- [3] J. Stewart, *Cálculo de una Variable*, Cengage Learning, 2012.

Regiones para el Cálculo de Áreas

Guía completa de geometría y cálculo integral

Elaborado por: ASISTENTE ACADÉMICO

6 de agosto de 2026

Índice

1. Introducción	2
2. Regiones geométricas elementales	2
3. Regiones definidas por funciones (Cálculo integral)	2
3.1. Región bajo una curva	3
3.2. Región entre dos curvas	3
3.3. Región en el eje y	3
3.4. Regiones con intersecciones	3
4. Regiones en coordenadas polares	3
5. Regiones paramétricas	3
6. Regiones compuestas	4
7. Ejemplo ilustrativo: área entre dos curvas	4
8. Conclusión	4

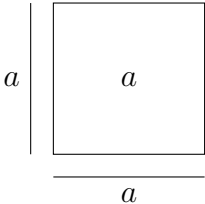
1. Introducción

El cálculo de áreas es una herramienta fundamental en matemáticas, física, ingeniería y ciencias aplicadas. Existen diversas regiones planas cuyas áreas pueden determinarse mediante fórmulas geométricas o mediante integración. Este documento recopila las regiones más importantes, clasificadas por su naturaleza y método de cálculo.

2. Regiones geométricas elementales

Estas regiones tienen fórmulas cerradas y son la base de la geometría euclidiana.

1. **Cuadrado** de lado a :

$$A = a^2$$


2. **Rectángulo** de base b y altura h :

$$A = b \cdot h$$

3. **Triángulo** de base b y altura h :

$$A = \frac{1}{2}bh$$

4. **Círculo** de radio r :

$$A = \pi r^2$$

5. **Sector circular** de radio r y ángulo θ (en radianes):

$$A = \frac{1}{2}r^2\theta$$

6. **Trapezio** de bases B y b , altura h :

$$A = \frac{B+b}{2} \cdot h$$

7. **Elipse** con semiejes a y b :

$$A = \pi ab$$

3. Regiones definidas por funciones (Cálculo integral)

Cuando la región está limitada por curvas, se usa la integral definida.

3.1. Región bajo una curva

Para una función $f(x) \geq 0$ en $[a, b]$:

$$A = \int_a^b f(x) dx$$

3.2. Región entre dos curvas

Si $f(x) \geq g(x)$ en $[a, b]$:

$$A = \int_a^b [f(x) - g(x)] dx$$

3.3. Región en el eje y

Si la región está limitada por funciones $x = f(y)$ y $x = g(y)$ con $f(y) \geq g(y)$ en $[c, d]$:

$$A = \int_c^d [f(y) - g(y)] dy$$

3.4. Regiones con intersecciones

Cuando las curvas se cruzan, se deben dividir los intervalos en los puntos de intersección.

4. Regiones en coordenadas polares

Para curvas dadas en polares $r = f(\theta)$, el área barrida entre α y β es:

$$A = \frac{1}{2} \int_{\alpha}^{\beta} [f(\theta)]^2 d\theta$$

Si hay dos curvas $r_1(\theta)$ y $r_2(\theta)$ con $r_1 \geq r_2$:

$$A = \frac{1}{2} \int_{\alpha}^{\beta} (r_1^2 - r_2^2) d\theta$$

5. Regiones paramétricas

Si una curva está dada paramétricamente por $x = x(t)$, $y = y(t)$ con $t \in [a, b]$ y la curva es cerrada o se recorre en sentido positivo, el área encerrada es:

$$A = \int_a^b y(t) x'(t) dt$$

o equivalentemente:

$$A = \frac{1}{2} \int_a^b (x(t)y'(t) - y(t)x'(t)) dt$$

6. Regiones compuestas

Muchas áreas reales son combinaciones de las anteriores. Por ejemplo:

- Corona circular: $A = \pi(R^2 - r^2)$
- Segmento parabólico: $A = \frac{2}{3}bh$
- Lúnulas y regiones entre arcos.

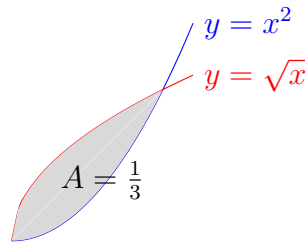
7. Ejemplo ilustrativo: área entre dos curvas

Calculemos el área entre $y = x^2$ y $y = \sqrt{x}$ en $[0, 1]$.

Puntos de intersección: $x^2 = \sqrt{x} \Rightarrow x^4 = x \Rightarrow x(x^3 - 1) = 0 \Rightarrow x = 0, 1$.

Como $\sqrt{x} \geq x^2$ en $[0, 1]$:

$$A = \int_0^1 (\sqrt{x} - x^2) dx = \left[\frac{2}{3}x^{3/2} - \frac{x^3}{3} \right]_0^1 = \frac{2}{3} - \frac{1}{3} = \frac{1}{3}$$



8. Conclusión

Conocer las regiones típicas y sus fórmulas permite abordar problemas de área de manera eficiente. La clave está en identificar la naturaleza de la región (geométrica, funcional, polar o paramétrica) y aplicar el método adecuado.

Comandos Básicos en Python

Uso de `input()`, `print()`, variables enteras y el comando `int()`

Departamento de Informática

6 de agosto de 2026

Índice

1. Introducción	2
2. El comando <code>print()</code>	2
2.1. Sintaxis básica	2
2.2. Salida con múltiples elementos	2
2.3. Formateo de cadenas	2
3. El comando <code>input()</code>	2
3.1. Sintaxis básica	3
3.2. Importante: <code>input()</code> siempre devuelve una cadena de texto	3
4. Variables numéricas enteras y operaciones	3
4.1. Declaración de variables enteras	3
4.2. Operadores aritméticos básicos	3
4.3. Ejemplos de operaciones	3
5. El comando <code>int()</code>	4
5.1. Convertir una cadena a entero	4
5.2. Convertir un número decimal a entero (truncamiento)	4
5.3. Uso combinado con <code>input()</code>	4
5.4. Manejo de errores	4
6. Ejemplos completos	4
6.1. Ejemplo 1: Calculadora básica	4
6.2. Ejemplo 2: Conversión de unidades	5
6.3. Ejemplo 3: Cálculo de promedio	5
7. Resumen y buenas prácticas	5
8. Ejercicios propuestos	5

1. Introducción

En este documento exploraremos cuatro conceptos fundamentales en la programación con Python:

- El comando `print()`, para mostrar información en pantalla.
- El comando `input()`, para recibir datos del usuario.
- Las variables numéricas de tipo entero (`int`) y las operaciones aritméticas básicas.
- El comando `int()`, para convertir datos a números enteros.

Estos elementos son la base para crear programas interactivos y realizar cálculos.

2. El comando `print()`

El comando `print()` se utiliza para mostrar mensajes, valores de variables o resultados de operaciones en la consola.

2.1. Sintaxis básica

```
1 print("Hola, mundo")
2 print(42)
3 print(3 + 5)
```

2.2. Salida con múltiples elementos

Podemos mostrar varios elementos separándolos por comas:

```
1 nombre = "Ana"
2 edad = 25
3 print("Nombre:", nombre, "Edad:", edad)
```

2.3. Formateo de cadenas

Existen varias formas de formatear la salida:

```
1 # Usando f-strings (recomendado)
2 print(f"El resultado es {resultado}")
3
4 # Usando el m todo format()
5 print("El resultado es {}".format(resultado))
6
7 # Usando el operador %
8 print("El resultado es %d" % resultado)
```

3. El comando `input()`

El comando `input()` permite que el programa reciba datos escritos por el usuario durante la ejecución.

3.1. Sintaxis básica

```
1 nombre = input(" Cmo te llamas? ")
2 print("Hola", nombre)
```

3.2. Importante: input() siempre devuelve una cadena de texto

Todos los datos ingresados con `input()` son de tipo `str` (cadena de caracteres), incluso si el usuario escribe números.

```
1 edad = input("Ingresa tu edad: ")
2 print(type(edad)) # Muestra: <class 'str'>
```

4. Variables numéricas enteras y operaciones

Las variables de tipo entero (`int`) almacenan números sin parte decimal. Con ellas podemos realizar operaciones aritméticas básicas.

4.1. Declaración de variables enteras

```
1 a = 10
2 b = 3
3 c = -5
```

4.2. Operadores aritméticos básicos

Operador	Descripción	Ejemplo
+	Suma	$10 + 3 = 13$
-	Resta	$10 - 3 = 7$
*	Multiplicación	$10 * 3 = 30$
/	División (real)	$10 / 3 = 3.333...$
//	División entera	$10 // 3 = 3$
%	Módulo (resto)	$10 \% 3 = 1$
**	Potencia	$10 ** 3 = 1000$

Cuadro 1: Operadores aritméticos en Python

4.3. Ejemplos de operaciones

```
1 x = 15
2 y = 4
3
4 suma = x + y
5 resta = x - y
6 multiplicacion = x * y
7 division_real = x / y
8 division_entera = x // y
9 resto = x % y
```

```

10 potencia = x ** y
11
12 print(f"Suma: {suma}")
13 print(f"Resta: {resta}")
14 print(f"Multiplicaci n: {multiplicacion}")
15 print(f"Divisi n real: {division_real}")
16 print(f"Divisi n entera: {division_entera}")
17 print(f"Resto: {resto}")
18 print(f"Potencia: {potencia}")

```

5. El comando int()

El comando `int()` se utiliza para convertir otros tipos de datos a números enteros.

5.1. Convertir una cadena a entero

```

1 numero_texto = "25"
2 numero_entero = int(numero_texto)
3 print(numero_entero + 5) # Resultado: 30

```

5.2. Convertir un número decimal a entero (truncamiento)

decimal = 7.89 entero = `int(decimal)` Resultado: 7 `print(entero)`

5.3. Uso combinado con input()

La combinación más común es:

```

1 edad = int(input("Ingresa tu edad: "))
2 anio_nacimiento = 2026 - edad
3 print(f"Naciste aproximadamente en el a o {anio_nacimiento}")

```

5.4. Manejo de errores

Es importante considerar que `int()` generará un error si la cadena no representa un número válido.

```

1 # Esto causar un error:
2 # numero = int("hola")
3
4 # Podemos manejar el error con try/except
5 try:
6     numero = int(input("Ingresa un n mero: "))
7     print(f"El n mero es {numero}")
8 except ValueError:
9     print("Error: Debes ingresar un n mero entero v lido.")

```

6. Ejemplos completos

6.1. Ejemplo 1: Calculadora básica

```

1 print("=== CALCULADORA BASICA ===")
2 num1 = int(input("Ingresa el primer numero: "))
3 num2 = int(input("Ingresa el segundo numero: "))
4
5 print(f"\n{num1} + {num2} = {num1 + num2}")
6 print(f"{num1} - {num2} = {num1 - num2}")
7 print(f"{num1} * {num2} = {num1 * num2}")
8 print(f"{num1} / {num2} = {num1 / num2}")
9 print(f"{num1} // {num2} = {num1 // num2}")
10 print(f"{num1} % {num2} = {num1 % num2}")
11 print(f"{num1} ** {num2} = {num1 ** num2}")

```

6.2. Ejemplo 2: Conversión de unidades

```

1 print("=== CONVERSION DE GRADOS ===")
2 celsius = int(input("Ingresa la temperatura en C : "))
3 fahrenheit = (celsius * 9/5) + 32
4 print(f"{celsius} C equivalen a {fahrenheit} F ")

```

6.3. Ejemplo 3: Cálculo de promedio

```

1 print("=== CALCULO DE PROMEDIO ===")
2 nota1 = int(input("Ingresa la primera nota: "))
3 nota2 = int(input("Ingresa la segunda nota: "))
4 nota3 = int(input("Ingresa la tercera nota: "))
5
6 promedio = (nota1 + nota2 + nota3) / 3
7 print(f"El promedio es: {promedio:.2f}") # :.2f muestra 2 decimales

```

7. Resumen y buenas prácticas

- **print():** Úsalo para mostrar información al usuario.
- **input():** Úsalo para recibir datos; siempre devuelve texto.
- **int():** Convierte texto o números decimales a enteros.
- **Operaciones:** Recuerda que la división / siempre devuelve un número decimal (float).
- **Seguridad:** Siempre valida la entrada del usuario para evitar errores.
- **Lectura:** Nombra tus variables de forma descriptiva (ej. edad, precio).

8. Ejercicios propuestos

1. Crea un programa que pida dos números enteros y muestre su suma, resta, multiplicación y división entera.
2. Escribe un programa que calcule el área de un rectángulo a partir de su base y altura (enteros).

3. Desarrolla un programa que pida un número de tres dígitos y muestre sus dígitos por separado (usando operaciones con enteros).
4. Crea un conversor de segundos a horas, minutos y segundos (usando división entera y módulo).

Regiones Básicas de la Geometría Plana y sus Áreas

Elaborado por: Asistente de IA

6 de agosto de 2026

Resumen

Este documento presenta una síntesis de las regiones planas fundamentales en geometría euclidiana: triángulo, cuadrado, rectángulo, paralelogramo, trapecio y círculo. Para cada figura se define su área y se ilustra con un gráfico realizado en TikZ, con el fin de ofrecer una referencia visual y algebraica para el estudio de la medición de superficies.

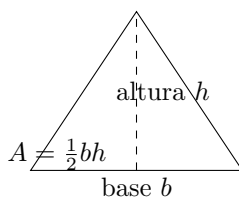
Introducción

El cálculo del área es una de las aplicaciones más importantes de la geometría. Conocer el área de figuras planas permite resolver problemas en diversas áreas como la arquitectura, la ingeniería y las ciencias naturales. A continuación se presentan las figuras más básicas y sus respectivas fórmulas.

Triángulo

El triángulo es un polígono de tres lados. Su área se calcula como la mitad del producto de la base por la altura:

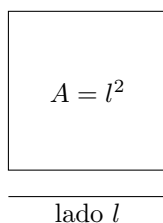
$$A = \frac{1}{2} \cdot b \cdot h$$



Cuadrado

El cuadrado tiene cuatro lados iguales y ángulos rectos. Su área es el lado al cuadrado:

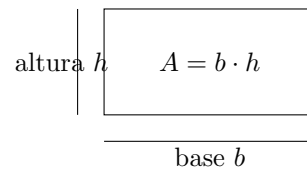
$$A = l^2$$



Rectángulo

El rectángulo tiene lados opuestos iguales y cuatro ángulos rectos. Su área es el producto de la base por la altura:

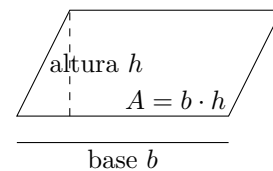
$$A = b \cdot h$$



Paralelogramo

El paralelogramo tiene lados opuestos paralelos e iguales. Su área es el producto de la base por la altura perpendicular:

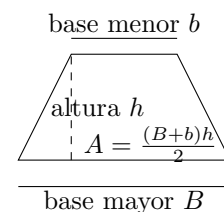
$$A = b \cdot h$$



Trapecio

El trapecio tiene al menos un par de lados paralelos (bases). Su área es el semiproducto de la suma de las bases por la altura:

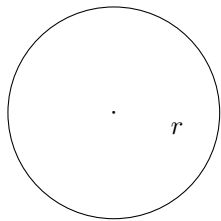
$$A = \frac{(B + b) \cdot h}{2}$$



Círculo

El círculo es la región delimitada por una circunferencia. Su área depende del radio:

$$A = \pi r^2$$



$$A = \pi r^2$$

Consideraciones finales

Estas seis figuras constituyen la base para el estudio de áreas en geometría plana. Con ellas es posible descomponer polígonos más complejos y aproximar regiones curvas mediante métodos de integración o discretización. La comprensión de estas fórmulas resulta esencial para cualquier estudiante de matemáticas, física o ingeniería.

hola mundo

mi nombre es:

```
¡Hola, mundo desde Google Colab!  
Python es increíble  
¡Estoy aprendiendo a programar!
```

Hola, soy maria
Tengo 20 años
Vivo en medellin
Mi lenguaje favorito es Phyton

hola, soy maria. tengo 20 años

hola, soy maria. tengo 20. vivo en la ciudad de medellin y programo en Phytton

Hola, soy Ana y tengo 25 años

[illegible]

```
# Python como calculadora
print("=== Operaciones Básicas ===")
print(f"15 / 3 = {15 / 3}")
```

```
=== Operaciones Básicas ===
15 / 3 = 5.0
```

```
# Python como calculadora
# potenciacion
print("=== Operaciones Básicas ===")
print(f"2 ** 8 = {2 ** 8}")

=== Operaciones Básicas ===
2 ** 8 = 256
```

```
# Python como calculadora
# el residuo
print("=== Operaciones Básicas ===")
print(f"17 % 5 = {17 % 5}")

=== Operaciones Básicas ===
17 % 5 = 2
```

```
# Crear un archivo Python en VS Code
# 1. Haz clic en "Abrir Visual Studio Code" arriba
# 2. Crea un nuevo archivo: archivo.py
# 3. Copia cualquiera de los ejercicios anteriores
# 4. Guarda el archivo y ejecútalo

print("¡Hola desde Visual Studio Code!")
nombre = input("¿Cuál es tu nombre? ")
print(f"¡Bienvenido a Python, {nombre}!")

¡Hola desde Visual Studio Code!
¿Cuál es tu nombre? Maria
¡Bienvenido a Python, Maria!
```

Comienza a programar o [generar](#) con IA.

```
print("¡cuanto nos da restar 5 menos 4!")
resta = input("5 - 4 = {5 - 4} ")
print(f"¡esta operacio da como resultado, {resta}!")

¡cuanto nos da restar 5 menos 4!
5 - 4 = {5 - 4} 1
¡esta operacio da como resultado, 1!
```

```
print("hola universo")
```

hola universo

```
def andres_raiz(x):  
    y = (3+x**(1/2))/((x-1)**(1/2)+3)  
    return y  
  
andres_raiz(1)
```

1.3333333333333333

```
def andres_dos(x,n):  
    y = (3+x**(0.5))/((x-1)**(0.5)+3)  
    w = f"{y:.{n}f}"  
    return w  
  
andres_dos(1,14)
```

'1.333333333333333'

```
import math  
  
def andres_tres(x,n):  
    y = (3+x**(0.5))/((x-1)**(0.5)+3)  
    w = f"{y:.{n}f}"  
    z = math.sqrt(2)  
    m = f"{z:.{n}f}"  
    return f"el valor de la función es: {w} y el valor de la raíz de 2 es: {m}"  
  
andres_tres(1,4)  
#math.sqrt(2)  
  
# print(f"{andres_tres(1,10)} y {math.sqrt(2)}")
```

'el valor de la función es: 1.3333 y el valor de la raíz de 2 es: 1.4142'

```
import math  
  
#print(2**(1/3))  
  
pi = math.pi  
#print(math.pi)  
print(f"{pi:.2f}")  
  
e = math.e  
#print(e)  
print(f"{e:.2f}")  
  
z = f"{2**(1/2):.2f}"  
#print(f"{z:.2f}")  
  
print(z)  
  
k = f"{3**(1/2):.2f}"  
#print(f"{k:.2f}")  
  
print(k)
```

3.14
2.72
1.41
1.73

```
import math  
  
#raíz cuadrada de 2  
z = f"{2**(1/2):.2f}"  
print(z)  
  
#raíz cuadrada de 3  
k = f"{3**(1/2):.2f}"  
print(k)  
  
#euler  
e = math.e  
print(f"{e:.2f}")  
  
#pi  
pi = math.pi  
print(f"{pi:.2f}")
```

1.41
1.73
2.72
3.14

```
import math  
  
def andres_una_linea(n):  
    return f"La raíz de 2 es: {z}. La raíz de 3 es: {k}. El número euler es: {e}. Finalmente, el número pi es: {pi}"  
  
n = 10  
  
#raíz cuadrada de 2  
z = f"{2**(1/2):.{n}f}"  
  
#raíz cuadrada de 3
```

```
k = f"{3**(1/2):.{n}f}"

#euler
e = math.e

#pi
pi = math.pi

andres_una_linea(n)

'La raíz de 2 es: 1.4142135624. La raíz de 3 es: 1.7320508076. El número euler es: 2.718281828459045. Finalmente, el número pi es: 3.141592653589793 '
```

```
def deepseek_operaciones(decimals):
    return [f"{2**0.5:.{decimals}f}", f"{3**0.5:.{decimals}f}", f"{2.718281828459045:.{decimals}f}", f"{3.141592653589793:.{decimals}f}"]

deepseek_operaciones(decimals=14)

['1.41421356237310',
'1.73205080756888',
'2.71828182845905',
'3.14159265358979']
```

Comienza a programar o [generar](#) con IA.

Geometría Plana: Cálculo de Áreas

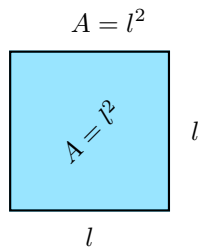
Regiones básicas y sus fórmulas

Introducción

La geometría plana estudia las figuras bidimensionales y sus propiedades. Una de las medidas más importantes es el *área*, que cuantifica la extensión de una superficie. En este artículo exploramos las regiones planas más básicas, presentando sus fórmulas de área y representaciones gráficas con el área interna resaltada en azul cielo.

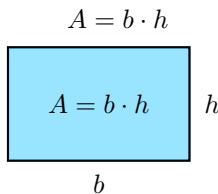
Cuadrado

El cuadrado es un polígono regular de cuatro lados iguales y cuatro ángulos rectos. Su área se calcula como el cuadrado de la longitud de su lado l :



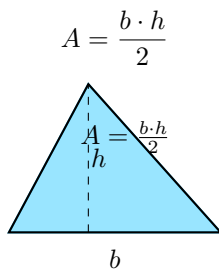
Rectángulo

El rectángulo tiene lados opuestos iguales y cuatro ángulos rectos. Su área es el producto de su base b por su altura h :



Triángulo

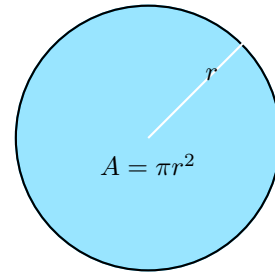
El triángulo es el polígono de tres lados. Su área se obtiene como la mitad del producto de la base b por la altura h :



Círculo

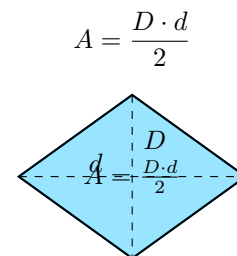
El círculo es la región delimitada por una circunferencia. Su área depende del radio r :

$$A = \pi r^2$$



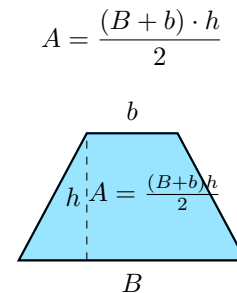
Rombo

El rombo es un paralelogramo con todos sus lados iguales. Su área puede calcularse como el producto de sus diagonales D (mayor) y d (menor) dividido por 2:



Trapezio

El trapezio es un cuadrilátero con un par de lados paralelos llamados bases (B mayor, b menor). Su área es:

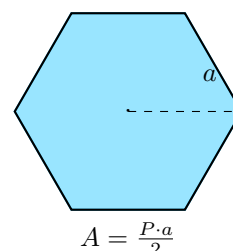


Polígono Regular

Un polígono regular tiene todos sus lados y ángulos iguales. Su área se calcula a partir del perímetro P y la apotema a :

$$A = \frac{P \cdot a}{2}$$

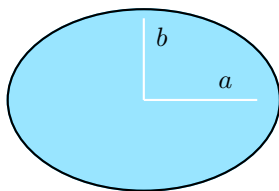
Para un hexágono regular de lado l : $A = \frac{3\sqrt{3}}{2} l^2$.



Elipse

La elipse es una curva cerrada que generaliza al círculo. Su área se calcula con los semiejes mayor a y menor b :

$$A = \pi \cdot a \cdot b$$



$$A = \pi ab$$

Conclusión

Hemos recorrido las figuras planas más fundamentales y sus respectivas fórmulas de área. Cada una de ellas tiene aplicaciones en disciplinas como la arquitectura, la ingeniería y las ciencias naturales. La representación gráfica con áreas sombreadas facilita la comprensión visual del concepto de área como la extensión de una región bidimensional.

Referencias

- [1] Weisstein, Eric W. "Área." *MathWorld*.
- [2] Stewart, J. *Cálculo de una Variable*. Cengage Learning.

¿Qué es GHAB y cómo se maneja?

Documento Breve

August 6, 2026

Abstract

Este artículo breve presenta una introducción a GHAB (Gestión Holística de Ambientes y Bienestar), un enfoque integrador para la administración de espacios físicos y digitales orientado al bienestar humano. Se describen sus principios fundamentales y se ofrece una guía práctica sobre su manejo.

1 Introducción

GHAB, acrónimo de Gestión Holística de Ambientes y Bienestar, es un paradigma emergente que fusiona conceptos de la psicología ambiental, el diseño universal y la administración de recursos para crear entornos que promuevan la salud física, mental y social de sus usuarios. Este enfoque se aplica en contextos que van desde oficinas y hogares hasta plataformas digitales y espacios públicos.

El término “holístico” es clave, ya que GHAB no se limita a aspectos estéticos o funcionales, sino que considera la interacción dinámica entre las personas, la tecnología y el ambiente físico.

2 Principios Fundamentales de GHAB

GHAB se sustenta en cinco pilares principales:

1. **Adaptabilidad:** Los ambientes deben ser flexibles para responder a las necesidades cambiantes de los usuarios.
2. **Sostenibilidad:** Se prioriza el uso eficiente de recursos y la reducción del impacto ambiental.
3. **Inclusividad:** Diseño accesible para personas con diversas capacidades y características.
4. **Tecnología integrada:** Uso de sensores e IA para monitorizar y ajustar condiciones en tiempo real.
5. **Bienestar comprobable:** Métricas basadas en evidencia para evaluar el impacto en la salud.

3 ¿Cómo se maneja GHAB?

El manejo de GHAB implica un proceso cíclico de diagnóstico, diseño, implementación y evaluación. A continuación, se detallan las fases principales:

3.1 Diagnóstico inicial

Se realiza un análisis del entorno actual y las necesidades de los usuarios. Esto incluye:

- Encuestas de percepción y bienestar.
- Medición de parámetros ambientales (luz, ruido, calidad del aire).
- Evaluación de flujos de trabajo y patrones de uso.

3.2 Diseño participativo

Los usuarios y partes interesadas colaboran en el diseño de soluciones. Se utilizan herramientas como el BIM (Building Information Modeling) y simulaciones digitales para prever el comportamiento del ambiente.

3.3 Implementación tecnológica

Se instalan sistemas de automatización y sensores IoT (Internet of Things) que permiten:

Ajuste dinámico = $f(\text{datos sensoriales, preferencias del usuario})$ ⁽¹⁾

Estos sistemas regulan iluminación, climatización, acústica y otros factores.

3.4 Monitoreo y mejora continua

Los datos recopilados se analizan periódicamente. Se generan reportes de rendimiento y bienestar, y se aplican ciclos de retroalimentación para refinar el ambiente.

3.5 Capacitación y cultura

Se fomenta la educación de los usuarios sobre el uso adecuado de los espacios y tecnologías, promoviendo hábitos que potencien el bienestar colectivo.

4 Herramientas y tecnologías comunes

- **Sensores ambientales:** Miden temperatura, humedad, CO₂ y luz.
- **Paneles de control inteligentes:** Interfaces que permiten a los usuarios modificar parámetros.
- **Analítica de datos:** Algoritmos de machine learning para predecir preferencias.
- **Plataformas colaborativas:** Apps para reportar incidencias y sugerir mejoras.

5 Desafíos y consideraciones

A pesar de sus beneficios, GHAB enfrenta retos como:

- Privacidad y seguridad de los datos personales.
- Costos iniciales de infraestructura tecnológica.
- Resistencia al cambio por parte de los usuarios.
- Necesidad de estándares y normativas claras.

6 Conclusiones

GHAB representa una evolución en la gestión de ambientes, colocando el bienestar humano en el centro del diseño y la operación. Su manejo eficaz requiere un enfoque multidisciplinario, tecnología avanzada y participación activa de los usuarios. Aunque aún en desarrollo, sus principios ofrecen un camino prometedor hacia espacios más saludables, eficientes y equitativos.

Usos de `print()` y `print(". ")` en Python

Documento Técnico

August 6, 2026

Abstract

Este documento explora los diversos usos de la función `print()` en Python, con un enfoque especial en la variante `print(". ")`. Se analizan desde los aspectos más básicos hasta aplicaciones avanzadas, incluyendo ejemplos prácticos que ilustran su utilidad en la depuración, visualización de datos y control de flujo.

1 Introducción

La función `print()` es una de las herramientas más fundamentales en Python. Permite mostrar información en la consola, lo que es esencial para la depuración, la interacción con el usuario y la presentación de resultados. Aunque parece simple, `print()` ofrece una gran flexibilidad a través de sus parámetros y su comportamiento.

En particular, `print(". ")` puede parecer trivial, pero tiene aplicaciones interesantes en la creación de barras de progreso, separadores visuales y patrones de texto.

2 Usos básicos de `print()`

2.1 Imprimir cadenas y variables

El uso más común es imprimir texto o el valor de variables:

```
1 nombre = "Ana"
2 print("Hola, ", nombre)
```

Esto produce: Hola, Ana

2.2 Imprimir múltiples elementos

Se pueden pasar varios argumentos separados por comas:

```
1 print("El resultado es", 42, "y el error es", 0.5)
```

2.3 El parámetro `sep`

Permite definir el separador entre los elementos:

```
1 print("uno", "dos", "tres", sep="-")
```

Salida: uno-dos-tres

2.4 El parámetro end

Controla lo que se imprime al final (por defecto es salto de línea):

```
1 print("Hola", end=" ")
2 print("Mundo")
```

Salida: Hola Mundo

3 Usos de print(". ")

Imprimir un punto puede parecer sencillo, pero tiene varios usos prácticos.

3.1 Separador visual

Se usa para crear líneas divisorias o marcas en la salida:

```
1 print(".")
2 print("-----")
3 print(".")
```

3.2 Indicador de progreso

En bucles largos, `print(".", end="")` imprime puntos en la misma línea para mostrar avance:

```
1 import time
2 for i in range(10):
3     time.sleep(0.5)
4     print(".", end="", flush=True)
5 print("    Completado    !")
```

3.3 Formateo de salida

Ayuda a crear patrones visuales como menús o tablas simples:

```
1 print("-----")
2 print("| Opci n 1 | Opci n 2 |")
3 print("-----")
```

3.4 Depuración visual

Se puede usar para marcar puntos específicos en la ejecución:

```
1 print(".")
2 x = 5 * 3
3 print(".")
4 print("Resultado:", x)
```

4 Uso avanzado: barras de progreso y efectos visuales

Combinando `print(".", end="")` con bucles y `flush=True`, se pueden crear barras de progreso simples:

```
1 import time
2 for i in range(1, 11):
3     time.sleep(0.3)
4     print(".", end="\r", flush=True)
5 print(" Proceso completado.")
```

También se puede usar para animar texto o crear menús interactivos en consola.

5 Conclusiones

La función `print()` en Python es mucho más versátil de lo que aparenta. Si bien `print(".")` parece insignificante, tiene aplicaciones útiles en visualización, depuración y retroalimentación al usuario. Dominar estos detalles permite escribir código más claro, interactivo y profesional.

References

- [1] Python Software Foundation. *Python Documentation: Built-in Functions*. <https://docs.python.org/3/library/functions.htmlprint>